Module 4: Methods in Java

Java Laboratory – Scenario-Based Problem Statements

Module Overview

Methods are the building blocks of modular programming and one of the most important concepts in Java. This module focuses on designing reusable, maintainable, and well-structured programs using user-defined methods. Students will learn to divide complex problems into smaller logical units, improving code readability, debugging, testing, and reusability.

Unlike simple mathematical exercises, the laboratory problems in this module are based on **real-world scenarios** from banking, healthcare, education, retail, transportation, hospitality, and business domains. Students are expected to identify reusable functionalities and implement them as independent methods.

Total Practice Problems: 20

Difficulty Distribution

- **Beginner:** 8 Problems
- **Intermediate:** 8 Problems
- **Advanced:** 4 Problems

Learning Objectives

After completing this module, students will be able to:

- Design modular Java applications using methods.
- Create methods with and without parameters.
- Create methods with and without return values.
- Apply method overloading.
- Implement recursion for suitable problems.
- Improve software quality through modular programming.
- Understand the importance of reusable code in enterprise applications.

Concepts Covered

- User-defined Methods
- Method Declaration
- Method Invocation
- Parameters and Arguments
- Return Values
- Void Methods
- Method Overloading
- Recursion
- Variable Scope
- Modular Programming

Beginner Level Problems

Problem 1: Student Academic Report Generator

Scenario

A faculty member wants to automate the process of preparing students' academic reports.

Problem Statement

Develop a Java application by creating separate methods to:

- Read student details.
- Read marks of five subjects.
- Calculate total marks.
- Calculate percentage.
- Assign grade.
- Display the final report card.

Each task must be implemented as a separate method.

Problem 2: Employee Salary Processing

Scenario

The HR department needs a simple application to generate employee salary slips.

Problem Statement

Create separate methods to:

- Accept employee details.
- Calculate HRA.
- Calculate DA.
- Calculate Tax.
- Calculate Net Salary.
- Display the salary slip.

Each salary component should be calculated in an independent method.

Problem 3: Restaurant Billing System

Scenario

A restaurant wants to automate bill generation for its customers.

Problem Statement

Design separate methods to:

- Read ordered items.
- Calculate subtotal.
- Calculate GST.
- Calculate service charges.
- Calculate final payable amount.
- Print the customer bill.

Problem 4: Electricity Bill Calculator

Scenario

An electricity department generates monthly bills for consumers.

Problem Statement

Develop a Java program with separate methods to:

- Read customer information.

- Calculate energy charges.
- Calculate fixed charges.
- Calculate taxes.
- Generate the final electricity bill.

Problem 5: Temperature Conversion Utility

Scenario

A weather application needs a utility to convert temperatures between different units.

Problem Statement

Create separate methods to convert:

- Celsius to Fahrenheit
- Fahrenheit to Celsius
- Celsius to Kelvin
- Kelvin to Celsius

Allow the user to select the desired conversion.

Problem 6: Banking Interest Calculator

Scenario

A bank offers different savings schemes.

Problem Statement

Create methods to:

- Read customer details.
- Calculate Simple Interest.
- Calculate Maturity Amount.
- Display account summary.

Use methods with parameters and return values.

Problem 7: Library Fine Calculator

Scenario

A college library charges fines for overdue books.

Problem Statement

Develop methods to:

- Read book details.
- Calculate overdue days.
- Calculate fine amount.
- Display borrowing summary.

Problem 8: Shopping Invoice Generator

Scenario

A retail store generates invoices for customers.

Problem Statement

Create separate methods to:

- Read product details.
- Calculate item totals.
- Calculate discounts.
- Calculate GST.
- Generate the final invoice.

Intermediate Level Problems

Problem 9: Hospital Patient Registration

Scenario

A hospital wants to automate patient registration and billing.

Problem Statement

Develop methods to:

- Register patient details.
- Assign consultation fee.
- Calculate medicine charges.
- Calculate total bill.
- Display patient summary.

Problem 10: Online Examination Result System

Scenario

An online learning platform generates examination results.

Problem Statement

Create methods to:

- Accept marks.
- Calculate total.
- Calculate percentage.
- Determine pass/fail status.
- Assign grade.
- Display result.

Problem 11: Hotel Room Booking System

Scenario

A hotel manages room bookings and customer bills.

Problem Statement

Implement separate methods for:

- Room booking.
- Room charge calculation.
- Food bill calculation.
- Tax calculation.

- Invoice generation.

Problem 12: Courier Charge Calculator

Scenario

A courier company calculates delivery charges based on parcel details.

Problem Statement

Create methods to:

- Accept parcel details.
- Calculate shipping charges.
- Calculate insurance charges.
- Display receipt.

Problem 13: Banking Transaction System

Scenario

A bank wants a modular application for customer transactions.

Problem Statement

Develop separate methods for:

- Deposit
- Withdraw
- Balance Inquiry
- Mini Statement

The `main()` method should only coordinate method calls.

Problem 14: Vehicle Service Center

Scenario

A vehicle service center generates service bills.

Problem Statement

Create methods to:

- Register vehicle.
- Calculate labour charges.
- Calculate spare part charges.
- Calculate GST.
- Print invoice.

Problem 15: Method Overloading – Unit Converter

Scenario

A scientific calculator provides multiple unit conversions.

Problem Statement

Use **method overloading** to implement conversions for:

- Kilometers to Miles
- Kilograms to Pounds
- Celsius to Fahrenheit
- Litres to Gallons

Use appropriate parameter types to invoke the correct overloaded method.

Problem 16: Airline Ticket Reservation

Scenario

An airline reservation system generates ticket information.

Problem Statement

Create methods to:

- Accept passenger details.
- Calculate ticket fare.

- Calculate taxes.
- Generate boarding pass.
- Display ticket summary.

Advanced Level Problems

Problem 17: University Admission Management System

Scenario

A university wants to automate its admission process.

Problem Statement

Develop a modular application with methods to:

- Register applicants.
- Validate eligibility.
- Calculate admission score.
- Assign department.
- Generate admission confirmation.

Every major task should be implemented as an independent method.

Problem 18: Smart ATM System

Scenario

An ATM application should provide multiple banking services.

Problem Statement

Implement a menu-driven application using separate methods for:

- Login validation.
- Cash withdrawal.
- Cash deposit.
- Balance inquiry.
- PIN change.
- Mini statement.

- Logout.

Ensure that the `main()` method only manages program flow.

Problem 19: Hospital Management System Using Modular Programming

Scenario

A hospital wants to simplify patient management.

Problem Statement

Create reusable methods for:

- Patient Registration.
- Doctor Allocation.
- Medicine Billing.
- Laboratory Charges.
- Final Bill Generation.
- Discharge Summary.

Design the application such that each operation is independent and reusable.

Problem 20: Payroll Management System

Scenario

A multinational company processes payroll for hundreds of employees every month.

Problem Statement

Develop a payroll application using modular programming.

Create separate methods for:

- Employee Registration.
- Salary Calculation.
- Allowance Calculation.
- Tax Deduction.

- Provident Fund Calculation.
- Net Salary Calculation.
- Payslip Generation.

The program should demonstrate the benefits of modular design through clear separation of responsibilities.

Challenge Problems

1. Develop a **Library Management System** using modular programming.
2. Create a **Hospital Appointment Booking System** with reusable methods.
3. Design an **Online Food Ordering System** where each operation is implemented as an independent method.
4. Build a **Travel Reservation System** with methods for booking, cancellation, fare calculation, and ticket generation.
5. Develop a **College ERP System** for student registration, fee calculation, attendance tracking, and report generation.
6. Create a **Retail Billing System** with separate methods for inventory lookup, discount calculation, taxation, and invoice printing.
7. Implement a **Scientific Calculator** using method overloading for different data types and operations.
8. Solve the **Tower of Hanoi** problem using recursion and compare it with an iterative solution in terms of readability and scalability.

Real-World Applications

The concepts covered in this module are widely used in:

- Banking software
- Hospital Management Systems
- University ERP Systems
- Hotel Reservation Systems
- Airline Booking Applications
- Payroll Management Software
- Inventory Management Systems
- Retail Billing Applications
- Enterprise Business Applications
- Government e-Governance Portals

Common Mistakes

Students should avoid the following:

- Writing all logic inside the `main()` method.
- Creating methods that perform multiple unrelated tasks.
- Choosing inappropriate method names.
- Passing incorrect arguments to methods.
- Ignoring return values from methods.
- Writing duplicate code instead of creating reusable methods.
- Overloading methods with ambiguous parameter lists.
- Missing or incorrect base cases in recursive methods.
- Excessive dependence on global/static variables instead of parameter passing.

Viva Questions

1. Why are methods called the building blocks of modular programming?
2. What are the advantages of using methods instead of writing all code in `main()`?
3. Differentiate between methods with parameters and methods without parameters.
4. Explain the difference between `void` methods and methods that return values.
5. What is method overloading? State the rules for valid method overloading.
6. Why can't methods be overloaded by changing only the return type?
7. What is recursion? Give two practical applications of recursive methods.
8. Explain Java's pass-by-value mechanism with an example.
9. How does modular programming improve software maintainability?
10. What characteristics define a well-designed method?

Expected Learning Outcomes

After completing this module, students will be able to:

- Design modular and reusable Java programs using user-defined methods.
- Break complex business problems into smaller, manageable functions.
- Pass data between methods using parameters and return values.
- Apply method overloading to provide multiple implementations of similar functionality.
- Use recursion appropriately for recursive problem-solving.
- Write clean, maintainable, and scalable Java applications that follow software engineering best practices.

Special Notes:

- Encourage students to identify reusable functionality before writing any code.
- Ask students to prepare a **method hierarchy diagram** showing how the `main()` method interacts with other methods.
- Emphasize the **Single Responsibility Principle (SRP)** by ensuring each method performs only one logical task.
- Discuss how modular programming simplifies debugging, testing, and maintenance, and relate these practices to enterprise Java frameworks such as Spring Boot, where applications are naturally organized into reusable service methods.